Message Passing Interface
MPI

1

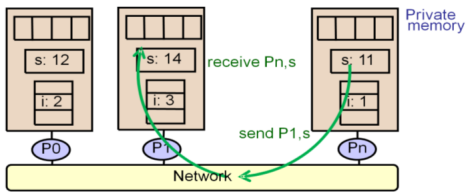
OUTLINE

- Programming model
- Parallélisme par passage de messages
- Gestion du contexte de communication et des processus

2

Message Passing
Programing model

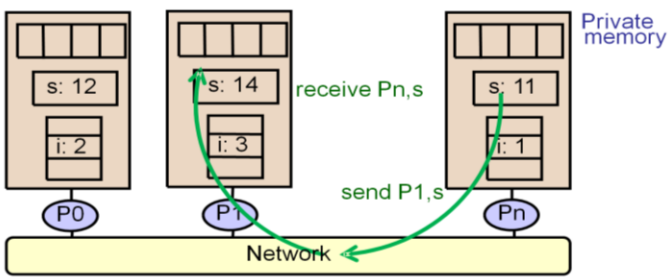
- Program consists of a collection of **named** processes
  - Number of processes almost always fixed at program startup time
  - Local address space per node – NO physically shared memory
  - Logically shared data is partitioned over local processes
- Processes communicate by explicit send/receive pairs
  - Coordination is implicit in every communication event
  - MPI (Message Passing Interface) is the most commonly used SW



3

Message Passing
Programing model

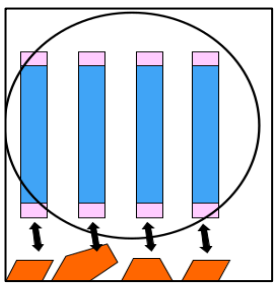
- One way of distributing work to multiple processes, which run *independently* and *concurrently* on either a single computer or a cluster of computers.
  - a single program runs on all the processes.
  - each process has a number or *rank*.
  - the value of the rank is used in the program to determine what each process will perform.



4

Message Passing
Programing model

- Single Program Mulitple Data (SPMD)
- Run the same program on P processing elements where P can be arbitrarily large.
- Use the rank … an ID ranging from 0 to (P-1) … to select between a set of tasks and to manage any shared data structures



This pattern is very general and has been used to support most (if not all) the algorithm strategy patterns

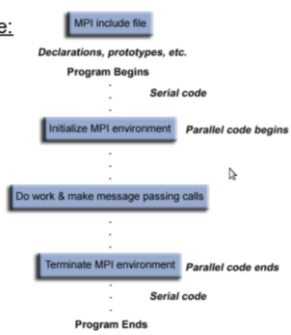
MPI programs almost always use this pattern … it is probably the most commonly used pattern in the history of parallel programming

5

Writing MPI Programs
MPI_INIT, MPI_FINALIZE

- **All MPI programs must include a header file. In C: mpi.h.**
- **All MPI programs must call MPI_INIT as the first MPI call. This establishes the MPI environment.**
- **All MPI programs must call MPI_FINALIZE as the last call, this exits MPI.**
- **Both MPI_INIT & FINALIZE returns MPI_SUCCESS if they are successfully exited**

MPI Program Structure:



6

Writing MPI Programs
MPI_INIT, MPI_FINALIZE

```
#include<stdio.h>
#include<mpi.h>

int main(int argc, char *argv[]){
int rank,size;

MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);

printf("Hello world, I am: %d of the nodes: %d\n", rank,size);

MPI_Finalize();
return 0;
}
```

- ***Only one invocation of MPI_INIT can occur in each program***
- ***MPI_FINALIZE terminates the MPI environment (no calls to MPI can be made after MPI_FINALIZE is called)***
- ***All non MPI routine are local; i.e. printf ("Welcome to MPI") runs on each processor***

7

Writing MPI Programs
MPI_COMM_WORLD

```
#include<stdio.h>
#include<mpi.h>

int main(int argc, char *argv[]){
int rank,size;

MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);

printf("Hello world, I am: %d of the nodes: %d\n", rank,size);

MPI_Finalize();
return 0;
}
```

- **How many processors are there?**
  MPI_COMM_SIZE
- **Who am I?**
  MPI_COMM_RANK

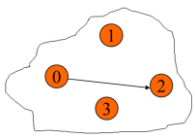
MPI_COMM_WORLD is the communicator, a group of processus

size returns the total number of processus, for size=4, ranks are 0,1,2,3 -

8

Writing MPI Programs
Sending and Receiving Messages

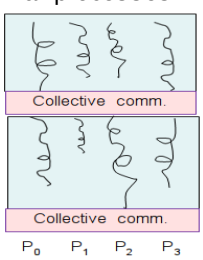
- Communication between processors involves:
  - identify sender and receiver
  - the type and amount of data that is being sent
  - how is the receiver identified?
- Communication types
  - Point to point communication: affects exactly two processors
- Collective communication: affects a group of processors in the communicator



9

Writing MPI Programs
Sending and Receiving Messages

- Many MPI applications have few (if any) sends and receives. They use the following very common pattern:
- Use the Single Program Multiple Data pattern
  - Each process maintains a local view of the global data
  - A problem broken down into phases each of which is composed of two sub-phases:
    - Compute on local view of data,
    - Communicate to update global view on all processes (collective communication),
    - Continue phases until complete



10

Writing MPI Programs
MPI_Send, MPI_Recv

- Communication between processors involves:
  - identify ***sender*** and ***receiver***
  - the ***type*** and ***amount*** of data that is being sent
  - how is the ***receiver identified***?
- Communication types
  - Point to point communication: affects exactly two processors
  - Collective communication: affects a group of processors in the communicator

11

Writing MPI Programs
MPI_Send, MPI_Recv

- int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)
  - ***source*** is the rank of the processor from which data will be accepted (this can be the rank of a specific processor or ***MPI_ANY_SOURCE***)
  - ***tag*** is an arbitrary number which can be used to distinguish different types of messages (from 0 to MPI_TAG_UB max=32767)
  - ***Status*** is implemented as structure with three fields: MPI_SOURCE, MPI_TAG, MPI_ERROR
  - ***Datatype***: MPI_CHAR, MPI_INT, MPI_FLOAT, MPI_DOUBLE, …

12

Writing MPI Programs

MPI_Send, MPI_Recv

```
#include "mpi.h"
main(int argc, char **argv)
{ int rank, size, tag, rc, i; MPI_Status status; char message[20];
  rc = MPI_Init(&argc, &argv);
  rc = MPI_Comm_size(MPI_COMM_WORLD, &size);
  rc = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  tag = 100;
      if(rank  == 0) { strcpy(message, "Hello, world");
      for (i=1; i<size; i++)
      rc = MPI_Send(message, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD);
      } else
      rc = MPI_Recv(message, 13, MPI_CHAR, 0, tag, MPI_COMM_WORLD,
      &status);
        printf( "node %d : %.13s\n", rank,message);
        rc = MPI_Finalize();
      }
```

```
node 0 : Hello, world
node 1 : Hello, world
node 2 : Hello, world
node 3 : Hello, world
node 4 : Hello, world
node 5 : Hello, world
```

13

Writing MPI Programs

MPI_Sendrecv

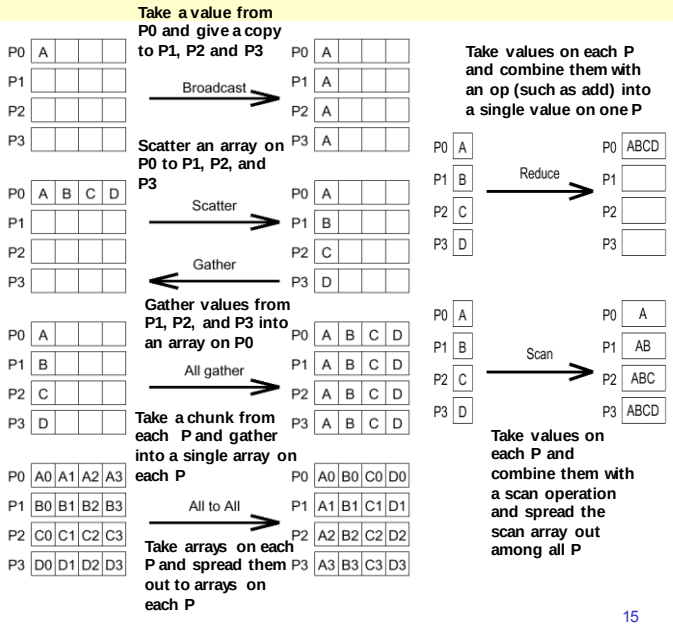
- MPI_Sendrecv function that both sends and receives a message.
- MPI_Sendrecv does not suffer from the circular deadlock problems of MPI_Send and MPI_Recv.
- You can think of MPI_Sendrecv as allowing data to travel for both send and receive simultaneously.
- The calling sequence of MPI_Sendrecv is the following:

```
int MPI_Sendrecv(void *sendbuf, int sendcount,
      MPI_Datatype senddatatype, int dest, int sendtag,
      void *recvbuf, int recvcount, MPI_Datatype recvdatatype,
      int source, int recvtag, MPI_Comm comm,
      MPI_Status *status)
```

14

Collective Communication & Computation

- *BARRIER*
- *BROADCAST*
- *REDUCTION*
- *PREFIX*
- *GATHER*
- *SCATTER*
- *ALL-to-ALL*



Writing MPI Programs

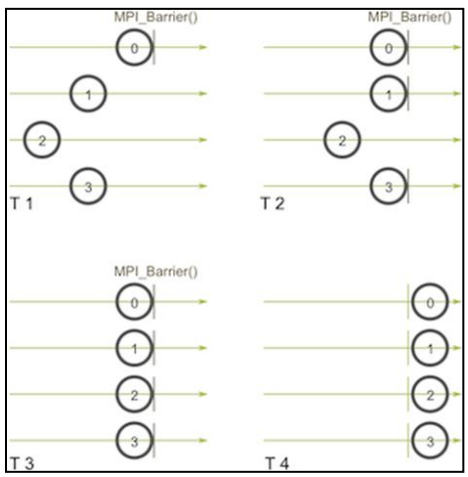
Barrier

- The barrier synchronization operation is performed in MPI using the MPI_Barrier function.

  int MPI_Barrier(MPI_Comm comm)

- The only argument of MPI_Barrier is the communicator that defines the group of processes that are synchronized.

- The call to MPI_Barrier returns only after all the processes in the group have called this function.

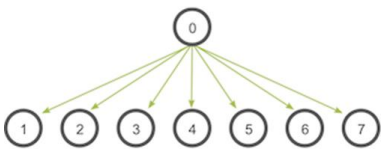


Collective Communication & Computation

BROADCAST

- The one-to-all broadcast operation is performed in MPI using the *MPI_Bcast* function.
- *MPI_Bcast* sends the data stored in the buffer *buf* of process *source* to all the other processes in the group.
- The data received by each process is stored in the buffer buf.
- The data that is broadcast consist of *count* entries of type *datatype*. The amount of data sent by the source process must be equal to the amount of data that is being received by each process; i.e., the *count* and *datatype* fields must match on all processes.

int MPI_Bcast(void *buf, int count, MPI_Datatype datatype, int source, MPI_Comm comm)



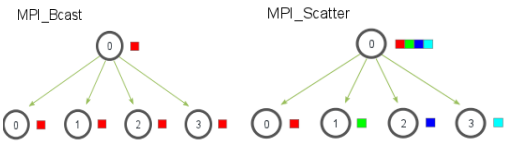
17

Collective Communication & Computation

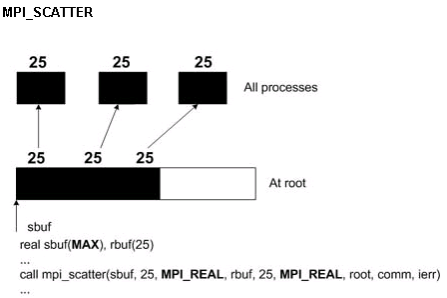
MPI_Scatter

- The scatter operation is performed in MPI using the *MPI_Scatter* function.
- The source process sends a different part of the send buffer *sendbuf* to each processes, including itself. The data that are received are stored in *recvbuf*.

int MPI_Scatter(void *sendbuf, int sendcount, MPI_Datatype senddatatype, void *recvbuf, int recvcount, MPI_Datatype recvdatatype, int source, MPI_Comm comm)



One-to-many communication pattern



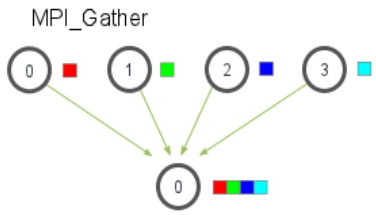
18

Collective Communication & Computation

MPI_Gather

- The gather operation is performed in MPI using the *MPI_Gather* function.
- Each process, including the target process, sends the data stored in the array *sendbuf* to the target process. As a result, if *p* is the number of processors in the communication comm, the target process receives a total of *p* buffers.
- The data is stored in the array *recvbuf* of the target process, in a rank order. That is, the data from process with *rank i* are stored in the *recvbuf* starting at location *i * sendcount* (assuming that the array *recvbuf* is of the same type as recvdatatype).

int MPI_Gather(void *sendbuf, int sendcount, MPI_Datatype senddatatype, void *recvbuf, int recvcount, MPI_Datatype recvdatatype, int target, MPI_Comm comm)



***many-to-one* communication pattern**

19

Collective Communication & Computation

MPI_Scatter, MPI_Gather

- Computes the average across all numbers in an array.
- Use MPI to divide work across processes, perform computation on subsets of data, and then aggregate the smaller pieces into the final answer.
  - Generate a random array of numbers on the root process (process 0).
  - Scatter the numbers to all processes, giving each process an equal amount of numbers.
  - Each process computes the average of their subset of the numbers.
  - Gather all averages to the root process.
  - The root process then computes the average of these numbers to get the final average.

```
mpirun -np 4 avg-gather-scatter 100
Avg of all elements is 0.518167
Avg computed across original data is 0.518167
```

20

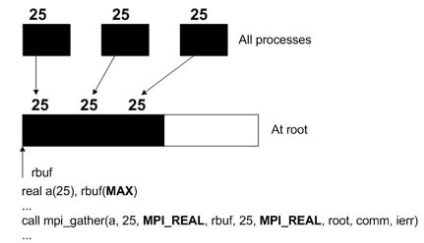
Collective Communication & Computation

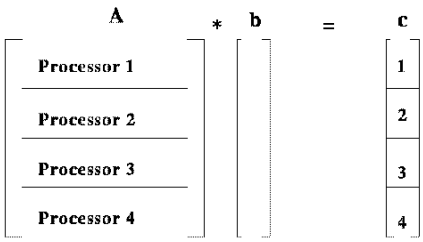
MPI_Scatter, MPI_Gather

- Multiplication of a matrix A, size 100x100, by a vector B of length 100. Four tasks, each task will work on its own chunk of 25 rows of A. B is the same for each task.
- The vector C will have 25 elements calculated by each task, stored in *cpart*. The *MPI_Gather* routine will retrieve *cpart* from each task and store the result in *ctotal*, which is the complete vector C.

```
double a[100,25],b[100],cpart[25],
ctotal[100];
 int root=0;
for(i=0;i<25;i++)
    { cpart[i]=0;
for(k=0;k<100;k++)
cpart[i]=cpart[i]+a[k,i]*b[k]; }
MPI_Gather(cpart,25,MPI_DOUBLE,ctot
al,25,MPI_DOUBLE,root,MPI_COMM_W
ORLD);
```

MPI_GATHER





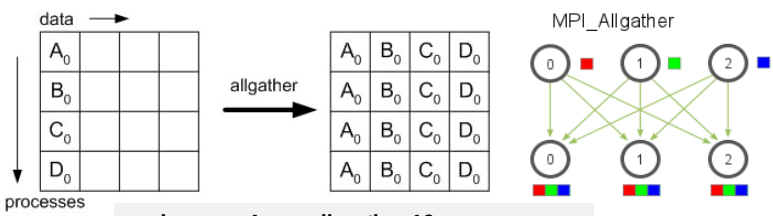
21

Collective Communication & Computation

MPI_Allgather

- MPI also provides the *MPI_Allgather* function in which the data are gathered to all the processes and not only at the target process.
- The meanings of the various parameters are similar to those for *MPI_Gather*; however, each process must now supply a *recvbuf* array that will store the gathered data.

int MPI_Allgather(void *sendbuf, int sendcount, MPI_Datatype senddatatype, void *recvbuf, int recvcount, MPI_Datatype recvdatatype, MPI_Comm comm)




```
mpirun -np 4 avg-all-gather 10
Avg of all elements from proc 0 is 0.504251
Avg of all elements from proc 1 is 0.504251
Avg of all elements from proc 2 is 0.504251
Avg of all elements from proc 3 is 0.504251
```

22

Collective Communication & Computation
REDUCE

- The *all-to-one reduction* operation is performed in MPI using the *MPI_Reduce* function.
- *MPI_Reduce* combines the elements stored in the buffer *sendbuf* of each process in the group using the operation specified in *op*, and returns the combined values in the buffer *recvbuf* of the process with *rank* target.
- Both the *sendbuf* and *recvbuf* must have the same number of *count* items of type *datatype*.
- All the processes must call *MPI_Reduce* with the same value for *count*, *datatype*, *op*, *target*, and *comm*.

int MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int target, MPI_Comm comm)

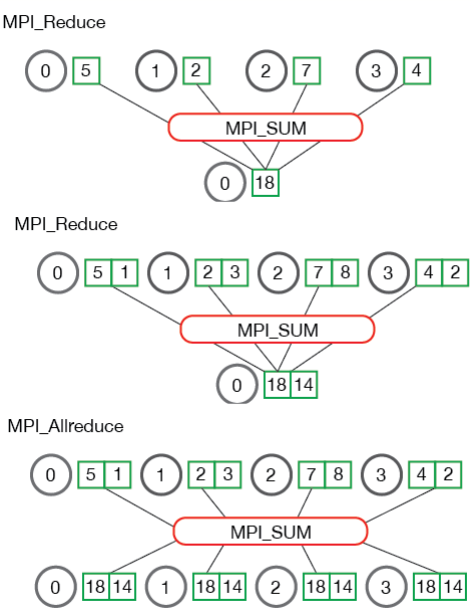
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)

23

Collective Communication & Computation
Reduce

Operation	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical AND
MPI_BAND	Bit-wise AND
MPI_LOR	Logical OR
MPI_BOR	Bit-wise OR
MPI_LXOR	Logical XOR
MPI_BXOR	Bit-wise XOR
MPI_MAXLOC	max-min value-location
MPI_MINLOC	min-min value-location

MPI_Allreduce* is the equivalent of doing *MPI_Reduce* followed by an *MPI_Bcast



Collective Communication & Computation

REDUCE

```
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);
// Sum the numbers locally
float local_sum = 0; int i;
for (i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
      world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT,  MPI_SUM, 0,
MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
        global_sum / (world_size * num_elements_per_proc));
}
```

Local sum for process 0 - 2.816110, avg = 0.704027
Total sum = 5.112439, avg = 0.639055
Local sum for process 1 - 2.296329, avg = 0.574082

25

References

- https://mpitutorial.com/tutorials/
- http://www.mpi-forum.org
- http://www.mcs.anl.gov/mpi
- https://computing.llnl.gov/tutorials/mpi
- https://rantahar.github.io/introduction-to-mpi/aio/index.html
- Parallel Programming in MPI and OpenMP, The Art of HPC, volume 2

26